App interaction quality: navigation, state, recovery and purposeful motion

Strengths and best use

Navigation, feedback, and motion work together to help people understand an application. Clear labels and reachable controls improve orientation; honest loading and recovery states explain progress; restrained transitions show continuity. This combined checklist turns the three reels into a task-based comparison rather than a collection of visual effects.

Build a reliable baseline

Choose three common tasks and test them on narrow and wide screens, with touch and a keyboard. Keep destination labels visible, indicate the active destination, and make focus obvious. Measure target size and spacing against the applicable accessibility guidance. A bottom bar should serve frequent destinations without covering page content or device insets. WCAG target size (https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html)

Represent loading, empty, error, success, and retry states explicitly. A skeleton should resemble the actual layout. Use optimistic updates only where failures can be explained and corrected. Disable duplicate submission while preserving server-side idempotency; a disabled button alone cannot prevent repeated requests. Offer meaningful undo when an action is reversible. Label form errors and keep entered values. Accessible forms (https://www.w3.org/WAI/tutorials/forms/)

Add motion with a purpose

Start with a dropdown, a list change, and an item-to-detail transition. Motion supports layout animation and shared layout identifiers; Radix provides a dropdown transform-origin variable. Use these capabilities to explain the interaction. Preserve focus, support fast reversal, and check behavior during repeated input. Motion layout (https://motion.dev/docs/react-layout-animations) - Radix dropdown (https://www.radix-ui.com/primitives/docs/components/dropdown-menu)

The reels' millisecond timings, stagger intervals, scale factors, and stretch percentages are starting points for tuning. They are not established universal optima. Respect reduced-motion preferences and offer a minimal-motion experience that retains the same information. Check the actual device frame timing before adding effects. Reduced motion (https://motion.dev/docs/react-use-reduced-motion) - Interaction animation (https://www.w3.org/WAI/WCAG22/Understanding/animation-from-interactions.html)

Compare the result

Record completion, wrong turns, first-tap accuracy, response delay, interruption behavior, and discomfort for the same tasks before and after changes. Include slow connections, failed requests, long labels, and zoom. Add shortcuts with discoverable alternatives. Apply predictive-back, OTP autofill, or platform gestures only where the platform supports them. Text wrapping, tabular numerals, safe-area spacing, and stable image dimensions can improve clarity without extra animation.

Reusable prompt

```
Review navigation, loading, recovery, and motion for these three tasks. Propose a small baseline, state what each change helps the user understand, and test touch, keyboard, focus, slow requests, interruption, duplicate submission, and reduced motion. Compare observed task outcomes before choosing the final details.
```

Project idea

Build an interaction comparison page with two navigation variants and three stateful components. Use fictional data and a worksheet of observations. Keep the original navigation, animation, and app-polish checklists as supporting references.

Original sources

- https://www.instagram.com/reel/DfRfBE_skb5/
- https://www.instagram.com/reel/DfXKRDQRBS_/
- https://www.instagram.com/reel/DfQczY4FE9Z/